

CS683 : Advanced Computer Architecture

Programming Assignment 1

Task 2 - Report

Submitted by

Ninad Narendra Kulkarni - 25M1227

Sam Navdeep Kolukula - 24b1303

Harshvardhan Parihar - 24b3980

Sairam Chari - 24b0937

CS ACA Programming Assignment 1

Task 2 - Report

Task 2A - Software Prefetching

1. Analyzing impact of Matrix size :

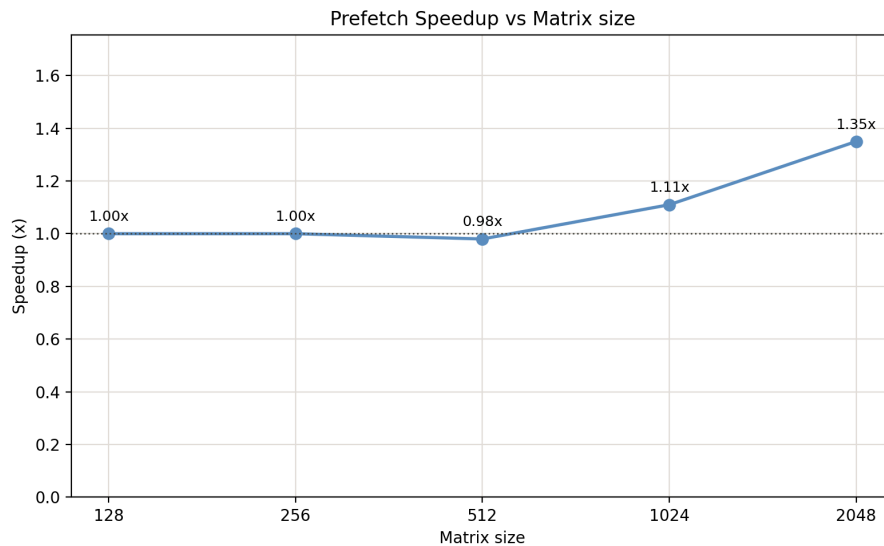


Figure : Matrix size vs Speedup (prefetch)

Just to start with, here we are prefetching with distance 16. The trend shows the speedup increased for larger matrices.

2. L1 misses and MPKI :

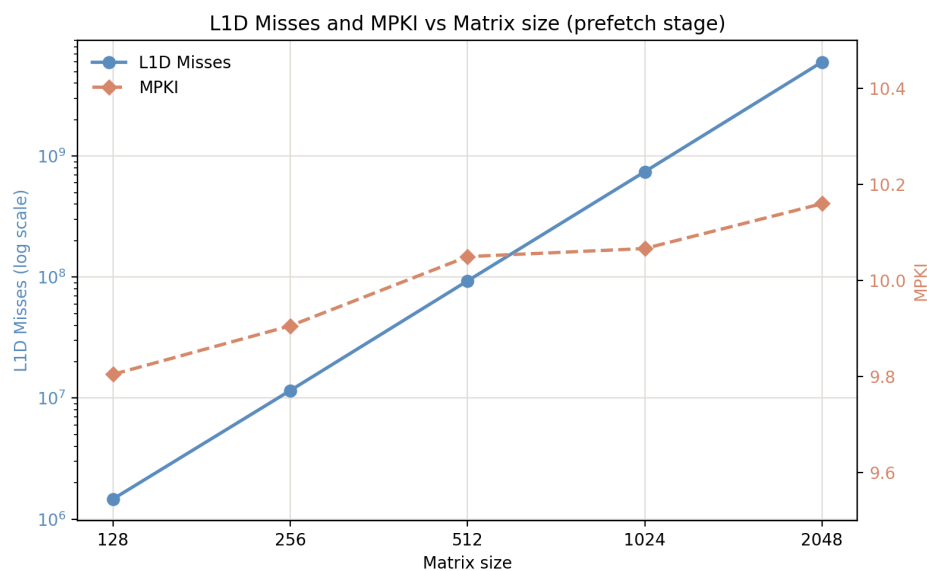


Figure : L1D misses vs Matrix size

L2 Misses :

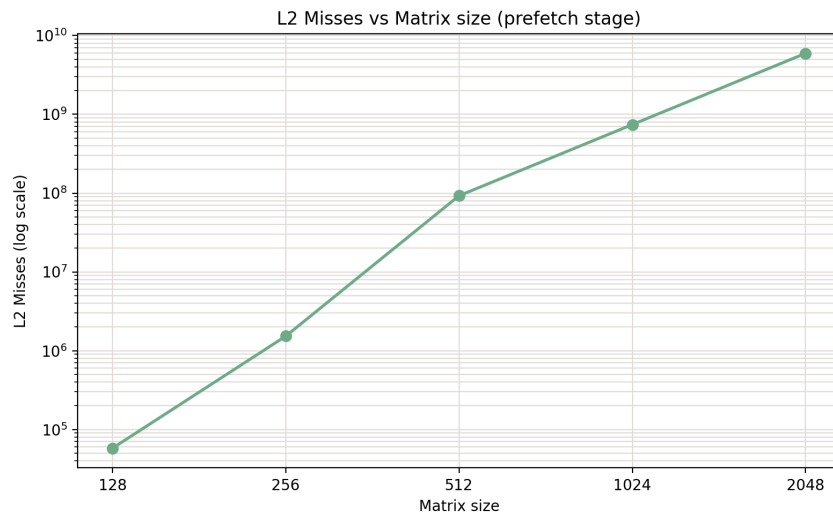


Figure : L2 misses vs Matrix size

As the matrix size increases, the cache becomes full and results into capacity misses that's why the L1 misses and thereby the MPKI increases.

3. Finding Speedup for SGEMM operation :

- Execution Time without Software Prefetching - Naive : 38.50 ms
- Execution Time with Software Prefetching - Prefetch : 35.43 ms

Speedup = 38.50 ms / 35.43 ms = **1.09x**

4. Prefetch Distance -

After trying out multiple prefetch distances, the optimal prefetch distance found was **128**. Which is giving **1.3x** speedup.

5. Effect Hardware Prefetcher - on SGEMM workload

Baseline with Prefetcher OFF - 40.16ms

Baseline with Prefetcher ON - 33.46ms

Speedup due to the Hardware Prefetchers = **1.2x**

Analysis -

- Speedup was observed for increasing matrix sizes. The most speedup was for 2048.
- The best prefetch distance is 128 for our setup.

Task 2B - SIMD programming -

Machine spec : My machine can use only the 128 and 256 wide SIMD instructions (512 are not supported)

1. SIMD instruction width - From task 1 of this assignment we found out that, using 256 wide SIMD instructions helps in reducing the number of instructions as well as it increases the speedup of the program. So we used the **256 wide SIMD instructions** for task 2 directly. We implemented a register tiled version with hardware vectorization.

2x4 Register Tiling : The outer loops compute the output matrix C in blocks of 2 rows and 4 columns. By giving 8 independent accumulator registers (c00 through c13) to this 2x4 block, the CPU reuses the loaded values from matrices A and B, and reduces the number of slow memory reads.

Vectorized Inner Loop : Along the shared K dimension, the steps are taken by 8 ($p += 8$). It loads 8 contiguous floats from A and 8 contiguous floats from B into 256-bit `__m256` registers, then executes `_mm256_fmadd_ps` to perform 8 simultaneous multiply-add operations per cycle.

Horizontal Reduction (`hsum_avx2`) : The SIMD loop accumulates data into 8 parallel lanes within a single vector register. Once the K loop finishes, the `hsum_avx2` helper function systematically folds those 8 parallel partial sums together into a single scalar float so it can be written to the output matrix.

Speedup - This approach gave around 10x speedup as compared to the naive code.

1. Speedup vs Matrix size plot :

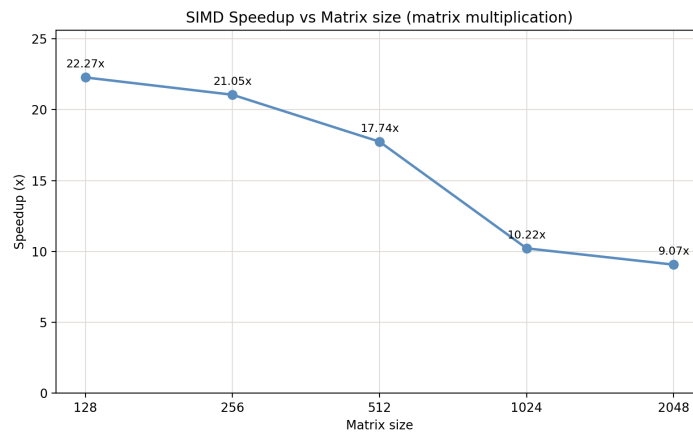


Figure : Speedup vs Matrix size (for SIMD)

As we can observe in the above graph, the speedup decreased with the increase in matrix size because the D cache is having capacity misses.

Peak Compute Efficiency (Sizes 128–256): The speedups (22.27x and 21.05x) are achieved because the entire matrix working set fits perfectly within the CPU's fast L1 and L2 caches. The hardware AVX2 execution units are saturated with data, allowing them to run close to theoretical peak math throughput without memory delays.

Memory limitation (Sizes 512–2048): As the matrices exceed L1 and L2 cache capacities, performance drops sharply (falling to 9.07x at size 2048). The CPU is forced to constantly fetch data from the slower L3 cache and main memory (DRAM). This latency causes the high-speed SIMD execution pipelines to stall while waiting for data to arrive, shifting the primary bottleneck from math execution to memory bandwidth.

3. Instruction count -

Calculating for 2048x2048 workload :

No. of Instructions for Prefetch -

-> Instructions: 820265736954

-> L1D Misses: 6032353938

-> L2 Misses: 8008624166

No of Instructions for SIMD -

-> Instructions: 318938909585

-> L1D Misses: 5337342694

-> L2 Misses: 4610487887

Analysis of Instruction count -

SIMD instructions process multiple data in single instruction hence we can observe the instruction count to be reduced.

4. Analyzing Speedup based on the Prefetching parameters

Adding software prefetching into SIMD -

The github README file asks to implement the tiling/blocking and then add prefetching on top of it.

We implemented the prefetching with different distances - it is found out that the prefetch distance 128 works well for this SIMD + Prefetch program (for 2048 size).

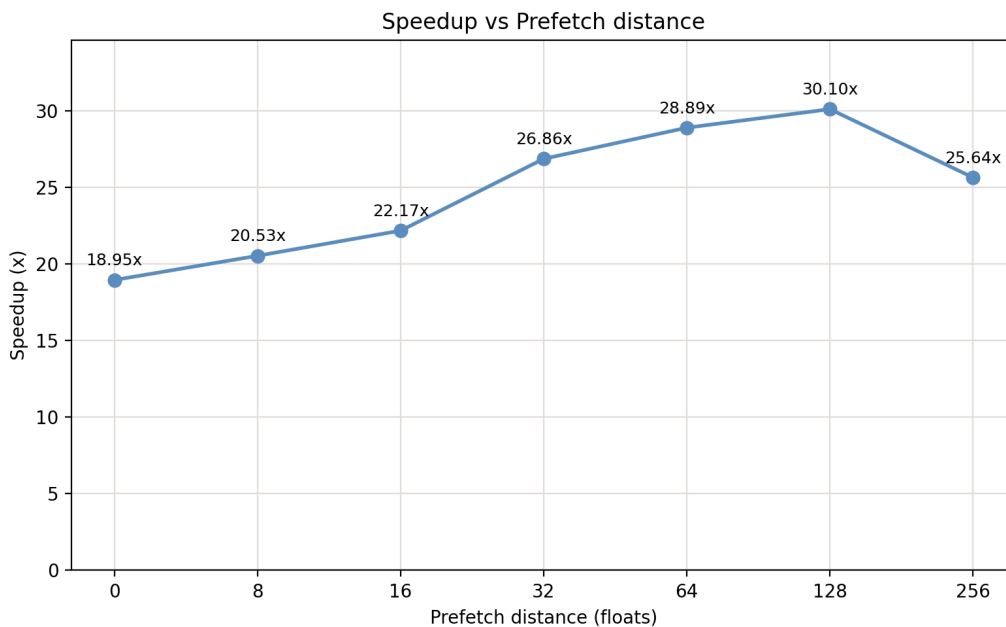


Figure : Speedup vs Prefetch Distance

Key Observations :

SIMD Only (Distance 0) : The 18.95x speedup is purely the result of AVX2 vectorization and register tiling. At a distance of 0, the prefetch instruction isn't doing anything.

Memory Latency Hiding (Distances 8–64) : Because the SIMD kernel computes fast, the CPU gets out of data quickly. Increasing the distance gives the memory controller the time it needs to fetch data from RAM into the L1 cache before the CPU actually asks for it.

Peak Performance (Distance 128) : Performance peaks at 30.10x because prefetching 128 floats (512 bytes) is the perfect architectural balance. It provides maximum latency hiding while keeping the prefetched data small enough to comfortably fit inside the L1 cache alongside the active computation blocks.

Cache Pollution (Distance 256) : The drop down to 25.64x happens because we are prefetching too far ahead. The hardware pulls future data in so early that it overwrites (evicts) the data the CPU is currently trying to use for its active operations. This is causing L1 cache misses to increase

The maximum speedup is achieved at a cache fill level of **512 bytes (or 8 cache lines) ahead of the current computation**. At this fill level, the prefetcher maintains a continuous runway of 128 floats inside the 32 KB L1 data cache.

Comparison Table between - Only Prefetch and SIMD (Instructions and Execution time)

		Matrix Size				
		128	256	512	1024	2048
Prefetch	Instructions	186100200	1544198584	12613675430	101988658916	820265736954
	Exec Time	2.098	17.908	150.088	1226.789	10644.165
SIMD + Prefetch	Instructions	83281106	638121314	5030992756	39990939455	318938909585
	Exec Time	0.094	0.817	8.187	178.936	1743.702

Comparison Table - Between only SIMD and SIMD + Prefetching (Speedup and MPKI Comcparison)

		Matrix Size				
		128	256	512	1024	2048
SIMD (No Prefetch)	Speedup	21.87	21.47	16.77	9.09	8.71
	MPKI	13.73	14.06	14.33	14.97	17.00
SIMD + Prefetch	Speedup	20.32	22.94	22.49	27.12	29.14
	MPKI	12.25	13.73	14.27	14.94	16.92

MPKI Comparison - between naive, SIMD only and SIMD + Prefetch

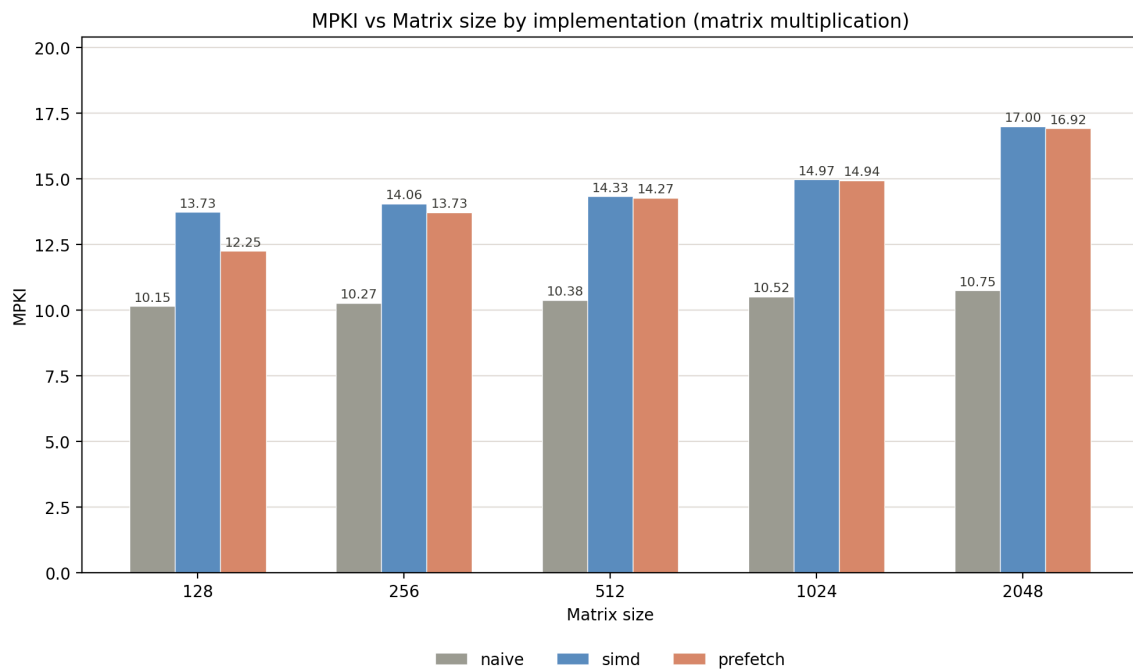


Figure : MPKI comparison between Naive, Only SIMD and SIMD + prefetch

Analysis of the plot -

lower MPKI (Misses Per Kilo-Instruction) does not necessarily indicate better performance. This becomes clear when cross-referencing the data with the execution speeds shown in the plot above

Naive : The naive implementation shows the lowest and most stable MPKI (~10.15 to 10.75) across all sizes in the plot representing MPKI. However, the Speedup values confirm it is vastly slower than the others (baseline 1.00x). Because MPKI normalizes misses against total instructions executed, the naive code's reliance on scalar math creates a massive instruction count. This inflates the denominator, lowering the MPKI metric and masking its poor memory efficiency.

SIMD : For the simd implementation in MPKI plot, MPKI climbs steadily from 13.73 at size 128 to 17.00 at size 2048. This directly explains the performance crash observed in Speedup plot, where the simd speedup lowers from 21.87x to 8.71x. Because AVX2 vectorization dramatically reduces the total instruction count (shrinking the denominator), the cache misses that occur when the matrix exceeds L2 capacity become highly exposed in the MPKI metric, resulting in severe CPU stalls.

Prefetch : At the 2048 matrix size, MPKI plot reveals that the simd and prefetch stages suffer from nearly identical MPKI levels (17.00 and 16.92, respectively). But, Speedup plot shows the prefetch implementation achieving a 29.14x speedup, completely crushing the simd implementation's 8.71x. This proves that software prefetching does not necessarily eliminate cache misses; instead, it hides the *latency* of those misses. The data is requested early enough that by the time the CPU needs it, it arrives, allowing the high-speed math pipelines to run without stalling.

Optimized Implementation : combination of all methods learnt till now

Techniques used in optimized version -

Cache Blocking (Tiling) : The outer loops divide the matrices into 64x64x256 blocks (BM, BN, BK). This ensures the active working data remains pinned in the CPU's fast 256 KB L2 cache, reducing slow round-trips to main memory.

Register Tiled Micro-Kernel : The inner loop computes a 2x4 tile of the output matrix C simultaneously. By dedicating eight 256-bit AVX2 registers (c00 through c13) as accumulators, this maximizes data reuse. A loaded value from matrix A or B is used multiple times across different accumulators before being evicted.

Loop Unrolling & SIMD (AVX2) : The innermost K-dimension loop is unrolled by a factor of two, processing 16 floats (64 bytes) per iteration using `_mm256_fmadd_ps` (Fused Multiply-Add). This gives the CPU's vector pipelines constantly and reduces loop branch overhead.

Software Prefetching : `_mm_prefetch` with `_MM_HINT_T0` explicitly triggers the memory controller to load data 32 floats (128 bytes) ahead of the current iteration directly into the L1 cache. This hides memory latency by ensuring data arrives just before the execution units need it.

Deferred Horizontal Reduction : SIMD registers accumulate partial sums vertically throughout the entire K-loop. The computationally expensive horizontal sum operation (`hsum_avx2`) is deferred until the very end, converting the 256-bit vector into a final scalar float for matrix C just once per element.

Fringe Handling : Clean-up scalar loops at the bottom process boundary elements (the "fringe") for matrices whose dimensions (M, N, K) are not perfect multiples of the block or micro-kernel sizes (8 in this case).

Finding Speedup for SGEMM operation :

- Execution Time without Software Prefetching - Naive : 38.50 ms
- Execution Time with Software Prefetching - Optimized : 19.35ms

Speedup = 38.50 ms / 19.35 ms = **1.98x**

Matrix Size vs Speedup : optimized implementation

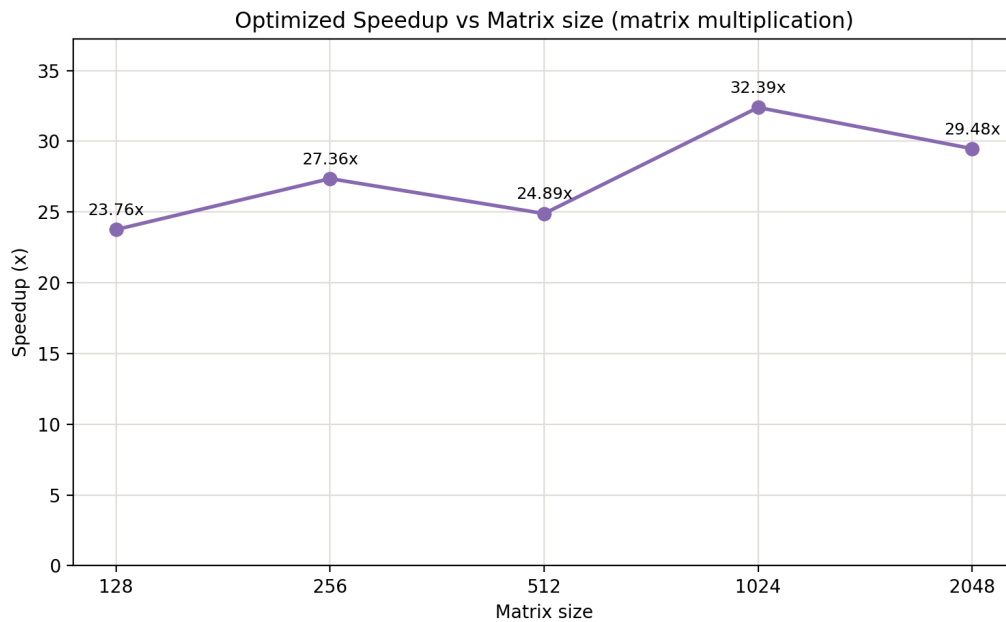


Figure : Matrix Size vs Speedup

Below is the table for the comparison of speedup of each implementation for different sizes of matrix.

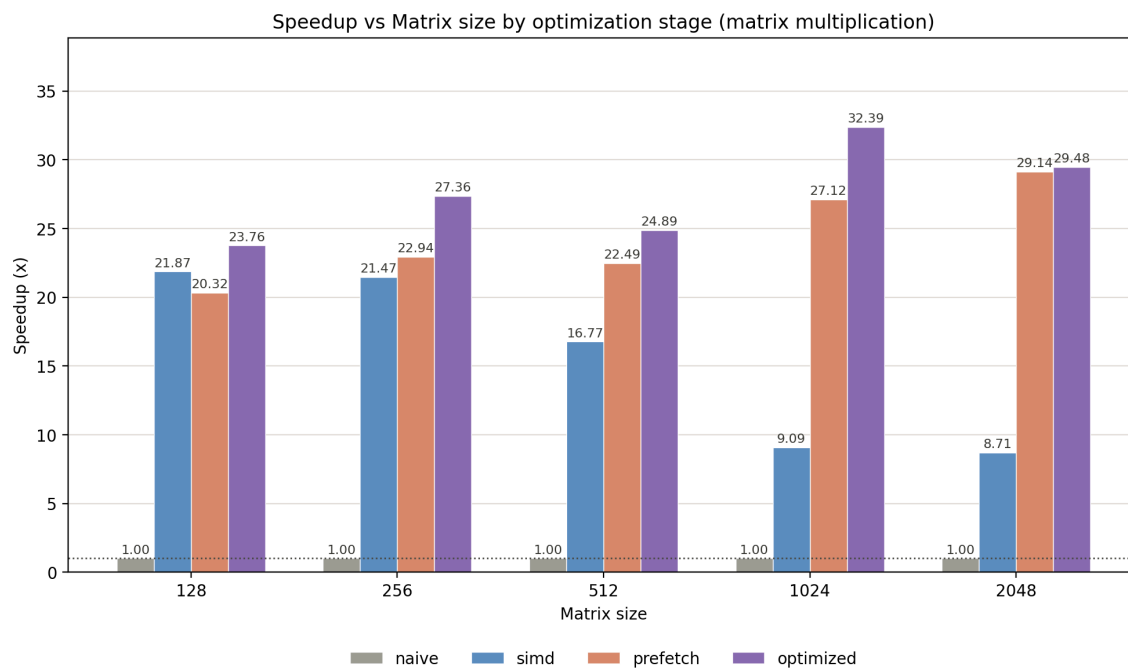


Figure : Overall Matrix size vs Speedup

Analysis : Optimized version

Peak Synergy (Optimized): The optimized stage layers all these architectural techniques combining AVX2 vectorization, aggressive cache blocking, and tuned software prefetching to achieve the highest overall performance. The performance sets at 32.39x at size 1024. The slight drop at 2048 (29.48x) suggests that even heavily optimized kernels eventually hit small degrades from cache contention or memory bandwidth limits at extreme scales.

Speedup for SGEMM application - 1.9x (compared to naive code)

```
Select ninad_ubuntu@sunilkul-T480: ~/MTech/cs_aca/assignment_1/temp_runs_t2
=====
Sweep complete. Detailed cache miss results saved to cache_misses_results.txt.
ninad_ubuntu@sunilkul-T480:~/MTech/cs_aca/assignment_1/temp_runs_t2$ OPT_SRC=src/matmul_baseline.cpp make llama-demo
bash llama/run_demo.sh
>>> llama.cpp matmul demo (tag b5731, 1 thread)
>>> resetting llama.cpp to pristine
>>> building STOCK llama-cli
>>> injecting student matmul_optimized
    patched ggml-cpu.c
    patched CMakeLists.txt
injection complete
>>> building STUDENT llama-cli
>>> running STOCK inference
>>> running STUDENT inference

===== RESULT =====
[ok] student matmul_optimized was actually used by ggml
[ok] student output matches stock output token-for-token (kernel is correct)

--- timings (llama.cpp native) ---
llama_perf_sampler_print:  sampling time =      0.92 ms /   69 runs (   0.01 ms per token, 75245.37 tokens per second)
llama_perf_context_print: prompt eval time =      0.65 ms /    5 tokens (   0.13 ms per token, 7739.94 tokens per second)
llama_perf_context_print:   eval time =    22.43 ms /   63 runs (   0.36 ms per token, 2808.36 tokens per second)
--- timings (student kernel) ---
llama_perf_sampler_print:  sampling time =      0.95 ms /   69 runs (   0.01 ms per token, 73015.87 tokens per second)
llama_perf_context_print: prompt eval time =      0.61 ms /    5 tokens (   0.12 ms per token, 8237.23 tokens per second)
llama_perf_context_print:   eval time =    19.35 ms /   63 runs (   0.31 ms per token, 3255.31 tokens per second)
=====
ninad_ubuntu@sunilkul-T480:~/MTech/cs_aca/assignment_1/temp_runs_t2$
```